```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from kneed import KneeLocator
from sklearn.preprocessing import StandardScaler

%pip install kneed

data = pd.read_csv("Iris.csv")
data.head()

print(data.isnull().sum())

features = [
"SepalLengthCm' ,
"SepalWidthCm' ,
"PetalLengthCm',
"PetalWidthCm'

X = data[features]

Ql = X.quantile(@.25)

Q3 = X.quantile(@.75)

IQR = Q3 - Ql

outliers = (
(X < (Q1 - 1.5 * IOQR)) |
(X > (Q3 + 1.5 * IQR)))

outlier_rows

data[outliers.any(axis=1) ]
print("Outlier rows:\n", outlier_rows)
data_cleaned = data[~outliers.any(axis=1) ].reset_index(drop=True)

print("Original dataset size:", data.shape[@])
print("Cleaned dataset size:", data_cleaned.shape[@])
```

```
X_cleaned = data_cleaned[ features ]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cleaned)

print("Scaled features (first 5 rows):\n", X_scaled[:5])

inertia=[]
K_range= range(1, 11)

for k in K_range:
kmeans = KMeans(
n_clusters=k, random_state=0, n_init=10 )
kmeans.fit(X_scaled)
inertia.append(kmeans.inertia_)

kl = KneeLocator(
K_range,
inertia,
curve="convex",
direction="decreasing"

)
optimal_k = kl.elbow
print("Optimal number of clusters (K):", optimal_k)

plt.figure(figsize=(8, 5))
plt.plot(K_range, inertia, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS (Inertia)")
plt.title("Elbow Method for Optimal K")
plt.show()

kmeans = KMeans(
n_clusters=optimal_k,
random_state=@,
n_init=10 )

clusters = kmeans.fit_predict(X_scaled)

data_cleaned['Cluster'] = clusters

print("Clustered Dataset (first 5 rows):\n")
```

```
print (data_cleaned.head())

plt.figure(figsize=(8, 6))
plt.scatter(
X_scaled[:, 0],
X_scaled[:, 1],
c=clusters,
cmap='viridis',
s=50
)
plt.xlabel("Sepal Length (scaled)")
plt.ylabel("Sepal Width (scaled)")
plt.title(f"K-Means Clusters of Iris Dataset (K={optimal_k})")
plt.show()
```